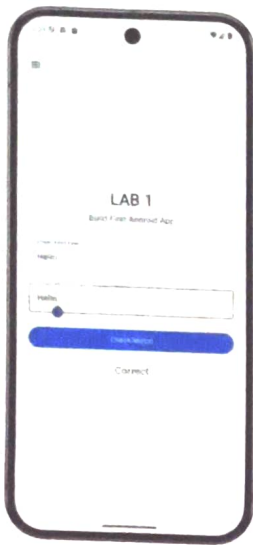


I N D E X

NAME: Makesh M STD.: CSE SEC.: 'C' ROLL NO.: 230701192 SUB.: MAD

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.		String Comparison	By	
2.		GUI Layer with Line button and Circle Button	By	
3.		App Navigation between two pages.	By	
4.		Activity and fragment Lifecycle with simple timer	By	
5.		To design a App Architecture.	By	
6.		App Architecture - Persistence Method.	By	
7.		Advanced Recycler - View Use Case	By	
8.		Internet Connectivity-Detection Using ConnectivityManager	By	
9.		Repository Pattern and Work Manager Implementation	By	
10.		App UI design Principles.	By	

Completed
B M



STRING COMPARISON

AIM:

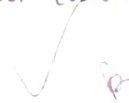
To check whether the first text and the second text are the same by considering uppercase and lowercase characters.

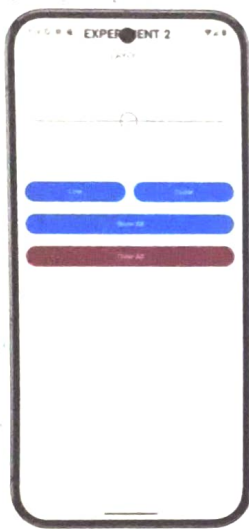
ALGORITHM:

- (1) Start the program.
- (2) Input the first text string.
- (3) Input the second text string.
- (4) Compare both strings character by character including case sensitivity.
- (5) If both strings are identical, print "Correct", otherwise print "Incorrect".

RESULT:

Thus, the program successfully checks whether two text strings are the same considering uppercase and lowercase characters.





GUI LAYOUT WITH LINE BUTTON AND CIRCLE BUTTON VISUALISATION

AIM:

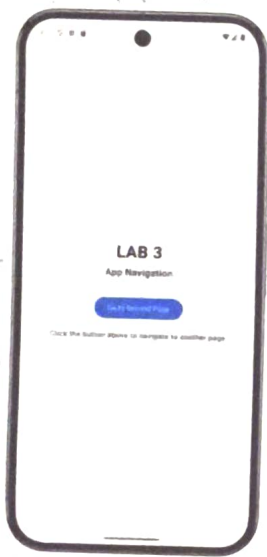
To design a graphical user interface with buttons to draw a Line and a circle, and to visualize them using Show All and Clear All buttons.

ALGORITHM:

- (1) Start the program and create a GUI window with a layout.
- (2) Add buttons for Line, Circle, Show All and Clear All.
- (3) When the Line button is pressed, generate or store the Line shape.
- (4) When the circle button is pressed, generate or store the circle shape.
- (5) Use Show All to display the shapes and Clear All to remove all shapes from the screen.

RESULT:

Thus, the GUI application successfully creates and visualizes line and circle shapes using buttons.



APP NAVIGATION BETWEEN TWO PAGES

AIM:

To implement navigation between the main page and the second page using buttons in an application.

ALGORITHM:

- (1) Start the application and create the main page interface
- (2) Add a button "Go to Second Page" on the main page.
- (3) When the button is clicked, navigate to the second page.
- (4) On the second page, add a button "Go to Main Page".
- (5) When the button is clicked, navigate back to the main page.

RESULT:

Thus, the application successfully performs navigation between the main page and the second page using buttons.



ACTIVITY AND FRAGMENT LIFECYCLE WITH SIMPLE TIMER

AIM :

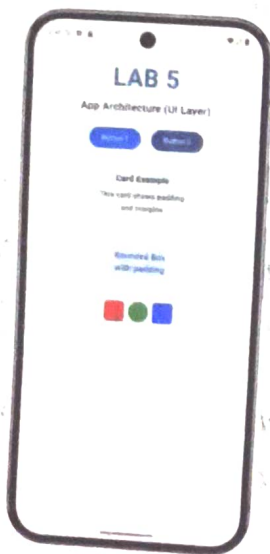
To demonstrate Activity and Fragment Lifecycle events and implement a simple timer with Start, Pause and Stop buttons

ALGORITHM :

- (1) Start the application and create an activity with a fragment and a timer display (00:00:00:00).
- (2) Display lifecycle events such as onCreate(), onStart() and onResume() when the activity starts
- (3) Add a Start button to begin the timer counting.
- (4) Add a Pause button to temporarily stop the timer
- (5) Add a Stop button to reset the timer to 00:00:00:00 and observe lifecycle events.

RESULT :

Thus, the program successfully demonstrates Activity and Fragment lifecycle events and implements a simple timer with Start, Pause and Stop Functions.



APP ARCHITECTURE - UI LAYER WITH BUTTONS AND CARD EXAMPLE

AIM:

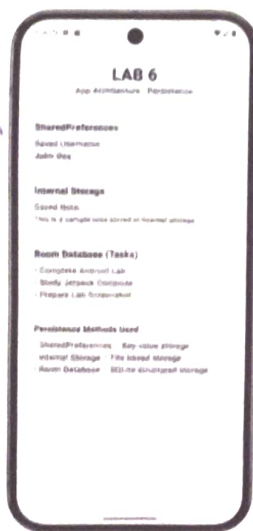
To design a simple UI layer using buttons and a card layout with a rounded box, padding and different colors

ALGORITHM:

- (1) Start the application and create the UI layout
- (2) Add Button1 and Button2 to the interface.
- (3) Create a card component with a rounded box and apply padding.
- (4) Apply different background colors such as red, green and blue to the UI elements
- (5) Display the UI layer with the designed buttons and colored card.

RESULT:

Thus, the program successfully demonstrates the UI layer of app architecture using buttons and a colored card layout.



APP ARCHITECTURE - PERSISTENCE METHODS

AIM:

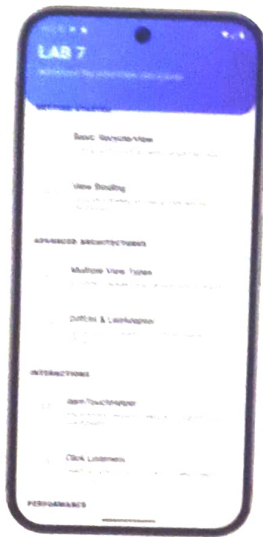
To demonstrate data persistence in an application using *SharedPreferences*, *Internal Storage* and *Room Database*

ALGORITHM:

- (1) Start the application and create the basic app architecture
- (2) Use *SharedPreferences* to store small Key-value data.
- (3) Use *Internal Storage* to save and retrieve data files inside the device.
- (4) Implement *Room Database* to store and manage tasks in a structured database.
- (5) Retrieve and display the stored data using the persistence methods.

RESULT:

Thus, the application successfully demonstrates data persistence using *SharedPreferences*, *Internal Storage* and *Room Database*.



ADVANCED RECYCLERVIEW USE CASES

AIM :

To implement Advanced RecyclerView features including multiple view types, interactions and performance optimization.

ALGORITHM :

- (1) Start the application and create a basic RecyclerView using ViewBinding.
- (2) Implement multiple view types and manage data using DiffUtil and ListAdapter.
- (3) Add user interactions using ItemTouchHelper and CheckListeners.
- (4) Improve performance by optimizing data updates and view recycling.
- (5) Display the list items efficiently using the Advanced RecyclerView Architecture.

RESULT :

Thus, the program successfully demonstrates advanced RecyclerView features with interactions and performance optimization.



INTERNET CONNECTIVITY DETECTION USING CONNECTIVITY MANAGER

AIM:

To detect internet connectivity and update the UI when the network status changes

ALGORITHM:

- (1) Start the application and check the internet connection
- (2) Use ConnectivityManager to monitor the network status.
- (3) Implement NetworkCallback to listen for real-time network changes.
- (4) Detect when the device gains or loses internet access.
- (5) Update the UI automatically based on the connectivity status

RESULT:

Thus, the application successfully detects internet connectivity changes and updates the UI automatically.



Repository Pattern And WorkManager Implementation

AIM :

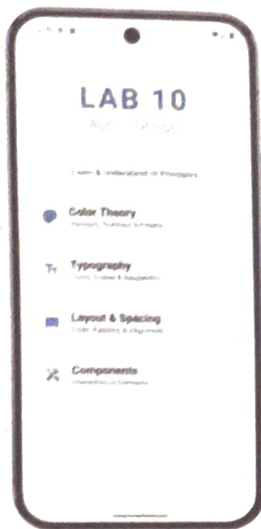
To implement the Repository Pattern and use WorkManager to perform background synchronization of data.

ALGORITHM :

- (1) Start the application and implement the Repository Pattern to manage data operations.
- (2) Use WorkManager to schedule background tasks.
- (3) Create a SyncWorker that runs in the background to add or sync new goals.
- (4) Execute the worker at specific time and date to perform synchronization.
- (5) Update the UI automatically using flows when new data is synced.

RESULT :

Thus, the application successfully performs background data synchronization using WorkManager and Repository Pattern.



APP UI DESIGN PRINCIPLES

AIM:

To understand and apply basic UI design principles in an application

ALGORITHM:

- (1) Start the application and design the basic user interface.
- (2) Apply color theory to choose suitable and consistent colors.
- (3) Use proper typography for readable and attractive text.
- (4) Maintain proper layout and spacing for better UI structure.
- (5) Add UI components such as buttons, cards and text elements.

RESULT:

Thus, the application successfully demonstrates basic UI design principles including color, typography, layout and components.